`hutil` — High-level file operations

Source code: Lib/shutil.py

The `shutil` module offers a number of high-level operations on files and collections of files. In particular, functions are provided which support file copying and removal. For operations on individual files, see also the `os` module.

Warning Even the higher-level file copying functions (`shutil.copy()`, `shutil.copy2()`) cannot copy all file metadata.

On POSIX platforms, this means that file owner and group are lost as well as ACLs. On Mac OS, the resource fork and other metadata are not used. This means that resources will be lost and file type and creator codes will not be correct. On Windows, file owners, ACLs and alternate data streams are not copied.

Directory and files operations

`shutil.`**`copyfileobj`**`(`*`fsrc, fdst`*`[,` *`length`*`])`

Copy the contents of the file-like object *fsrc* to the file-like object *fdst*. The integer *length*, if given, is the buffer size. In particular, a negative *length* value means to copy the data without looping over the source data in chunks; by default the data is read in chunks to avoid uncontrolled memory consumption. Note that if the current file position of the *fsrc* object is not 0, only the contents from the current file position to the end of the file will be copied.

`copyfileobj()` will *not* guarantee that the destination stream has been flushed on completion of the copy. If you want to read from the destination at the completion of the copy operation (for example, reading the contents of a temporary file that has

been copied from a HTTP stream), you must ensure that you have called `flush()` or `close()` on the file-like object before attempting to read the destination file.

```
shutil.copyfile(src, dst, *, follow_symlinks=True)
```

Copy the contents (no metadata) of the file named *src* to a file named *dst* and return *dst* in the most efficient way possible. *src* and *dst* are path-like objects or path names given as strings.

dst must be the complete target file name; look at `copy()` for a copy that accepts a target directory path. If *src* and *dst* specify the same file, `SameFileError` is raised.

The destination location must be writable; otherwise, an `OSError` exception will be raised. If *dst* already exists, it will be replaced. Special files such as character or block devices and pipes cannot be copied with this function.

If *follow_symlinks* is false and *src* is a symbolic link, a new symbolic link will be created instead of copying the file *src* points to.

Raises an auditing event `shutil.copyfile` with arguments `src` `dst`.

Changed in version 3.3: `IOError` used to be raised instead of `OSError`. Added *follow_symlinks* argument. Now returns *dst*.

Changed in version 3.4: Raise `SameFileError` instead of `Error`. Since the former is a subclass of the latter, this change is backward compatible.

Changed in version 3.8: Platform-specific fast-copy syscalls may be used internally in order to copy the file more efficiently. See Platform-dependent efficient copy operations section.

exception shutil.**SpecialFileError**

This exception is raised when `copyfile()` or `copytree()` attempt to copy a named pipe.

Added in version 2.7.

exception shutil.**SameFileError**

This exception is raised if source and destination in `copyfile()` are the same file.

Added in version 3.4.

shutil.**copymode**(*src*, *dst*, ***, *follow_symlinks=True*)

Copy the permission bits from *src* to *dst*. The file contents, owner, and group are unaffected. *src* and *dst* are path-like objects or path names given as strings. If *follow_symlinks* is false, and both *src* and *dst* are symbolic links, `copymode()` will attempt to modify the mode of *dst* itself (rather than the file it points to). This functionality is not available on every platform; please see `copystat()` for more information. If `copymode()` cannot modify symbolic links on the local platform, and it is asked to do so, it will do nothing and return.

Raises an auditing event `shutil.copymode` with arguments `src` `dst`.

Changed in version 3.3: Added *follow_symlinks* argument.

shutil.**copystat**(*src*, *dst*, ***, *follow_symlinks=True*)

Copy the permission bits, last access time, last modification time, and flags from *src* to *dst*. On Linux, `copystat()` also copies the “extended attributes” where

possible. The file contents, owner, and group are unaffected. *src* and *dst* are path-like objects or path names given as strings.

If *follow_symlinks* is false, and *src* and *dst* both refer to symbolic links, `copystat()` will operate on the symbolic links themselves rather than the files the symbolic links refer to—reading the information from the *src* symbolic link, and writing the information to the *dst* symbolic link.

Note Not all platforms provide the ability to examine and modify symbolic links. Python itself can tell you what functionality is locally available.

- If is , can modify the permission bits of a symbolic link.
- If is , can modify the last access and modification times of a symbolic link.
- If is , can modify the flags of a symbolic link. (is not available on all platforms.)

On platforms where some or all of this functionality is unavailable, when asked to modify a symbolic link, `copystat()` will copy everything it can. `copystat()` never returns failure.

Please see `os.supports_follow_symlinks` for more information.

Raises an auditing event `shutil.copystat` with arguments `src` `dst`.

Changed in version 3.3: Added *follow_symlinks* argument and support for Linux extended attributes.

```
shutil.copy(src, dst, *, follow_symlinks=True)
```

Copies the file *src* to the file or directory *dst*. *src* and *dst* should be path-like objects or strings. If *dst* specifies a directory, the file will be copied into *dst* using the base filename from *src*. If *dst* specifies a file that already exists, it will be replaced. Returns the path to the newly created file.

If *follow_symlinks* is false, and *src* is a symbolic link, *dst* will be created as a symbolic link. If *follow_symlinks* is true and *src* is a symbolic link, *dst* will be a copy of the file *src* refers to.

`copy()` copies the file data and the file's permission mode (see `os.chmod()`). Other metadata, like the file's creation and modification times, is not preserved. To preserve all file metadata from the original, use `copy2()` instead.

Raises an auditing event `shutil.copyfile` with arguments `src`, `dst`.

Raises an auditing event `shutil.copymode` with arguments `src`, `dst`.

Changed in version 3.3: Added *follow_symlinks* argument. Now returns path to the newly created file.

Changed in version 3.8: Platform-specific fast-copy syscalls may be used internally in order to copy the file more efficiently. See Platform-dependent efficient copy operations section.

shutil.**copy2**(*src*, *dst*, *, *follow_symlinks=True*)

Identical to copy() except that copy2() also attempts to preserve file metadata.

When *follow_symlinks* is false, and *src* is a symbolic link, copy2() attempts to copy all metadata from the *src* symbolic link to the newly created *dst* symbolic link. However, this functionality is not available on all platforms. On platforms where some or all of this functionality is unavailable, copy2() will preserve all the metadata it can; copy2() never raises an exception because it cannot preserve file metadata.

copy2() uses copystat() to copy the file metadata. Please see copystat() for more information about platform support for modifying symbolic link metadata.

Raises an auditing event `shutil.copyfile` with arguments `src`, `dst`.

Raises an auditing event `shutil.copystat` with arguments `src`, `dst`.

Changed in version 3.3: Added *follow_symlinks* argument, try to copy extended file system attributes too (currently Linux only). Now returns path to the newly created file.

Changed in version 3.8: Platform-specific fast-copy syscalls may be used internally in order to copy the file more efficiently. See Platform-dependent efficient copy operations section.

shutil.**ignore_patterns**(**patterns*)

This factory function creates a function that can be used as a callable for `copytree()`'s *ignore* argument, ignoring files and directories that match one of the glob-style *patterns* provided. See the example below.

```
shutil.copytree(src, dst, symlinks=False, ignore=None,
copy_function=copy2, ignore_dangling_symlinks=False,
dirs_exist_ok=False)
```

Recursively copy an entire directory tree rooted at *src* to a directory named *dst* and return the destination directory. All intermediate directories needed to contain *dst* will also be created by default.

Permissions and times of directories are copied with `copystat()`, individual files are copied using `copy2()`.

If *symlinks* is true, symbolic links in the source tree are represented as symbolic links in the new tree and the metadata of the original links will be copied as far as the platform allows; if false or omitted, the contents and metadata of the linked files are copied to the new tree.

When *symlinks* is false, if the file pointed to by the symlink doesn't exist, an exception will be added in the list of errors raised in an `Error` exception at the end of the copy process. You can set the optional *ignore_dangling_symlinks* flag to true if you want to silence this exception. Notice that this option has no effect on platforms that don't support `os.symlink()`.

If *ignore* is given, it must be a callable that will receive as its arguments the directory being visited by `copytree()`, and a list of its contents, as returned by `os.listdir()`. Since `copytree()` is called recursively, the *ignore* callable will be called once for each directory that is copied. The callable must return a sequence

of directory and file names relative to the current directory (i.e. a subset of the items in its second argument); these names will then be ignored in the copy process. `ignore_patterns()` can be used to create such a callable that ignores names based on glob-style patterns.

If exception(s) occur, an `Error` is raised with a list of reasons.

If *copy_function* is given, it must be a callable that will be used to copy each file. It will be called with the source path and the destination path as arguments. By default, `copy2()` is used, but any function that supports the same signature (like `copy()`) can be used.

If *dirs_exist_ok* is false (the default) and *dst* already exists, a `FileExistsError` is raised. If *dirs_exist_ok* is true, the copying operation will continue if it encounters existing directories, and files within the *dst* tree will be overwritten by corresponding files from the *src* tree.

Raises an auditing event `shutil.copytree` with arguments `src` `dst`.

Changed in version 3.2: Added the *copy_function* argument to be able to provide a custom copy function. Added the *ignore_dangling_symlinks* argument to silence dangling symlinks errors when *symlinks* is false.

Changed in version 3.3: Copy metadata when *symlinks* is false. Now returns *dst*.

Changed in version 3.8: Platform-specific fast-copy syscalls may be used internally in order to copy the file more efficiently. See Platform-dependent efficient copy operations section.

Changed in version 3.8: Added the *dirs_exist_ok* parameter.

```
shutil.rmtree(path, ignore_errors=False, onerror=None, *,
onexc=None, dir_fd=None)
```

Delete an entire directory tree; *path* must point to a directory (but not a symbolic link to a directory). If *ignore_errors* is true, errors resulting from failed removals will be ignored; if false or omitted, such errors are handled by calling a handler specified by *onexc* or *onerror* or, if both are omitted, exceptions are propagated to the caller.

If *path* is a symbolic link to a directory followed by a trailing path separator, such as `'link/'`, the operating system resolves the link, so `rmtree()` deletes the contents of the directory it points to and then fails to remove *path* itself. `pathlib.Path` strips trailing separators, so `rmtree(Path('link/'))` refuses the link as usual.

This function can support paths relative to directory descriptors.

Note On platforms that support the necessary fd-based functions a symlink attack resistant version of `rmtree()` is used by default. On other platforms, the `rmtree()` implementation is susceptible to a symlink attack: given proper timing and circumstances, attackers can manipulate symlinks on the filesystem to delete files they wouldn't be able to access otherwise. Applications can use the `rmtree.avoids_symlink_attacks` function attribute to determine which case applies.

If *onexc* is provided, it must be a callable that accepts three parameters: *function*, *path*, and *excinfo*.

The first parameter, *function*, is the function which raised the exception; it depends on the platform and implementation. The second parameter, *path*, will be the path

name passed to *function*. The third parameter, *excinfo*, is the exception that was raised. Exceptions raised by *onexc* will not be caught.

The deprecated *onerror* is similar to *onexc*, except that the third parameter it receives is the tuple returned from `sys.exc_info()`.

See also rmtree example for an example of handling the removal of a directory tree that contains read-only files.

Raises an auditing event `shutil.rmtree` with arguments `path`, `dir_fd`.

Changed in version 3.3: Added a symlink attack resistant version that is used automatically if platform supports fd-based functions.

Changed in version 3.8: On Windows, will no longer delete the contents of a directory junction before removing the junction.

Changed in version 3.11: Added the *dir_fd* parameter.

Changed in version 3.12: Added the *onexc* parameter, deprecated *onerror*.

Changed in version 3.13: `rmtree()` now ignores `FileNotFoundError` exceptions for all but the top-level path. Exceptions other than `OSError` and subclasses of `OSError` are now always propagated to the caller.

`rmtree.`**`avoids_symlink_attacks`**

Indicates whether the current platform and implementation provides a symlink attack resistant version of `rmtree()`. Currently this is only true for platforms supporting fd-based directory access functions.

Added in version 3.3.

```
shutil.move(src, dst, copy_function=copy2)
```

Recursively move a file or directory (*src*) to another location and return the destination.

If *dst* is an existing directory or a symlink to a directory, then *src* is moved inside that directory. The destination path in that directory must not already exist.

If *dst* already exists but is not a directory, it may be overwritten depending on `os.rename()` semantics.

`os.rename()` is preferably used internally when *src* and the destination are on the same filesystem. In case `os.rename()` fails due to `OSError` (e.g. the user has write permission to the destination file but not to its parent directory), this method falls back to using *copy_function*, in which case *src* is copied to the destination using *copy_function* and then removed.

In case of symlinks, a new symlink pointing to the target of *src* will be created in or as the destination, and *src* will be removed.

If *copy_function* is given, it must be a callable that takes two arguments, *src* and the destination, and will be used to copy *src* to the destination if `os.rename()` cannot be used. If the source is a directory, `copytree()` is called, passing it the *copy_function*.

The default *copy_function* is `copy2()`. Using `copy()` as the *copy_function* allows the move to succeed when it is not possible to also copy the metadata, at the expense of not copying any of the metadata.

Raises an auditing event `shutil.move` with arguments `src`, `dst`.

Changed in version 3.3: Added explicit symlink handling for foreign filesystems, thus adapting it to the behavior of GNU's **mv**. Now returns *dst*.

Changed in version 3.5: Added the *copy_function* keyword argument.

Changed in version 3.8: Platform-specific fast-copy syscalls may be used internally in order to copy the file more efficiently. See Platform-dependent efficient copy operations section.

Changed in version 3.9: Accepts a path-like object for both *src* and *dst*.

`shutil.`**`disk_usage`**`(`*`path`*`)`

Return disk usage statistics about the given path as a named tuple with the attributes *total*, *used* and *free*, which are the amount of total, used and free space, in bytes. *path* may be a file or a directory.

Note On Unix filesystems, *path* must point to a path within a **mounted** filesystem partition. On those platforms, CPython doesn't attempt to retrieve disk usage information from non-mounted filesystems.

Added in version 3.3.

Changed in version 3.8: On Windows, *path* can now be a file or directory.

Availability: Unix, Windows.

```
shutil.chown(path, user=None, group=None, *, dir_fd=None,
follow_symlinks=True)
```

Change owner *user* and/or *group* of the given *path*.

user can be a system user name or a uid; the same applies to *group*. At least one argument is required.

See also `os.chown()`, the underlying function.

Raises an auditing event `shutil.chown` with arguments `path`, `user`, `group`.

Availability: Unix.

Added in version 3.3.

Changed in version 3.13: Added *dir_fd* and *follow_symlinks* parameters.

```
shutil.which(cmd, mode=os.F_OK | os.X_OK, path=None)
```

Return the path to an executable which would be run if the given *cmd* was called. If no *cmd* would be called, return `None`.

mode is a permission mask passed to `os.access()`, by default determining if the file exists and is executable.

path is a "`PATH` string" specifying the directories to look in, delimited by `os.pathsep`. When no *path* is specified, the `PATH` environment variable is read from `os.environ`, falling back to `os.defpath` if it is not set.

If *cmd* contains a directory component, `which()` only checks the specified path directly and does not search the directories listed in *path* or in the system's `PATH` environment variable.

On Windows, the current directory is prepended to the *path* if *mode* does not include `os.X_OK`. When the *mode* does include `os.X_OK`, the Windows API `NeedCurrentDirectoryForExePathW` will be consulted to determine if the current directory should be prepended to *path*. To avoid consulting the current working directory for executables: set the environment variable `NoDefaultCurrentDirectoryInExePath`.

Also on Windows, the `PATHEXT` environment variable is used to resolve commands that may not already include an extension. For example, if you call `shutil.which("python")`, `which()` will search `PATHEXT` to know that it should look for `python.exe` within the *path* directories. For example, on Windows:

```
shutil.which("python")
'C:\\Python33\\python.EXE'
```

This is also applied when *cmd* is a path that contains a directory component:

```
shutil.which("C:\\Python33\\python")
'C:\\Python33\\python.EXE'
```

Added in version 3.3.

Changed in version 3.8: The bytes type is now accepted. If *cmd* type is `bytes`, the result type is also `bytes`.

Changed in version 3.12: On Windows, the current directory is no longer prepended to the search path if *mode* includes `os.X_OK` and WinAPI `NeedCurrentDirectoryForExePathW(cmd)` is false, else the current directory is prepended even if it is already in the search path; `PATHEXT` is used now even when *cmd* includes a directory component or ends with an extension that is in `PATHEXT`; and filenames that have no extension can now be found.

exception shutil.**Error**

Subclass of OSError collecting exceptions raised during a multi-file operation. For copytree(), the exception argument is a list of 3-tuples (*srcname*, *dstname*, *exception*).

Platform-dependent efficient copy operations

Starting from Python 3.8, all functions involving a file copy (copyfile(), copy(), copy2(), copytree(), and move()) may use platform-specific "fast-copy" syscalls in order to copy the file more efficiently (see bpo-33671). "fast-copy" means that the copying operation occurs within the kernel, avoiding the use of userspace buffers in Python as in "`outfd.write(infd.read())`".

On macOS fcopyfile is used to copy the file content (not metadata).

On Linux os.copy_file_range() or os.sendfile() is used.

On Solaris os.sendfile() is used.

On Windows shutil.copyfile() uses a bigger default buffer size (1 MiB instead of 64 KiB) and a memoryview()-based variant of shutil.copyfileobj() is used, which still reads and writes in a loop. shutil.copy2() uses the native CopyFile2 call on Windows, which is the most efficient method, supports copy-on-write, and preserves metadata.

If the fast-copy operation fails and no data was written in the destination file then shutil will silently fall back to less efficient copyfileobj() function internally.

Changed in version 3.8.

Changed in version 3.14: Solaris now uses os.sendfile().

Changed in version 3.14: Copy-on-write or server-side copy may be used internally via os.copy_file_range() on supported Linux filesystems.

copytree example

An example that uses the ignore_patterns() helper:

```
from shutil import copytree, ignore_patterns

copytree(source, destination, ignore=ignore_patterns('*.pyc', 'tmp*'))
```

This will copy everything except .pyc files and files or directories whose name starts with tmp.

Another example that uses the *ignore* argument to add a logging call:

```
from shutil import copytree
import logging

def _logpath(path, names):
    logging.info('Working in %s', path)
    return []    # nothing will be ignored

copytree(source, destination, ignore=_logpath)
```

rmtree example

This example shows how to remove a directory tree on Windows where some of the files have their read-only bit set. It uses the onexc callback to clear the readonly bit and reattempt the remove. Any subsequent failure will propagate.

```
import os, stat
import shutil

def remove_readonly(func, path, _):
    "Clear the readonly bit and reattempt the removal"
    os.chmod(path, stat.S_IWRITE)
    func(path)

shutil.rmtree(directory, onexc=remove_readonly)
```

Archiving operations

Added in version 3.2.

Changed in version 3.5: Added support for the *xztar* format.

High-level utilities to create and read compressed and archived files are also provided. They rely on the `zipfile` and `tarfile` modules.

`shutil.`**`make_archive`**`(`*`base_name`*`, `*`format`*`[, `*`root_dir`*`[, `*`base_dir`*`[, `*`verbose`*`[, `*`dry_run`*`[, `*`owner`*`[, `*`group`*`[, `*`logger`*`]]]]]])`

> Create an archive file (such as zip or tar) and return its name.
>
> *base_name* is a string or path-like object specifying the name of the file to create, including the path, minus any format-specific extension.

format is the archive format: one of “zip” (if the `zlib` module is available), “tar”, “gztar” (if the `zlib` module is available), “bztar” (if the `bz2` module is available), “xztar” (if the `lzma` module is available), or “zstdtar” (if the `compression.zstd` module is available).

root_dir is a string or path-like object specifying a directory that will be the root directory of the archive, all paths in the archive will be relative to it; for example, we typically chdir into *root_dir* before creating the archive.

base_dir is a string or path-like object specifying a directory where we start archiving from; i.e. *base_dir* will be the common prefix of all files and directories in the archive. *base_dir* must be given relative to *root_dir*. See Archiving example with base_dir for how to use *base_dir* and *root_dir* together.

root_dir and *base_dir* both default to the current directory.

If *dry_run* is true, no archive is created, but the operations that would be executed are logged to *logger*.

owner and *group* are used when creating a tar archive. By default, uses the current owner and group.

logger must be an object compatible with **PEP 282**, usually an instance of `logging.Logger`.

The *verbose* argument is unused and deprecated.

Raises an auditing event `shutil.make_archive` with arguments `base_name`, `format`, `root_dir`, `base_dir`.

Note This function is not thread-safe when custom archivers registered with `register_archive_format()` do not support the *root_dir* argument. In this case it temporarily changes the current working directory of the process to *root_dir* to perform archiving.

Changed in version 3.8: The modern pax (POSIX.1-2001) format is now used instead of the legacy GNU format for archives created with `format="tar"`.

Changed in version 3.10.6: This function is now made thread-safe during creation of standard `.zip` and tar archives.

Changed in version 3.15: Accepts a path-like object for *base_name*, *root_dir* and *base_dir*.

`shutil.get_archive_formats()`

Return a list of supported formats for archiving. Each element of the returned sequence is a tuple `(name, description)`.

By default `shutil` provides these formats:

- *zip*: ZIP file (if the `zlib` module is available).
- *tar*: Uncompressed tar file. Uses POSIX.1-2001 pax format for new archives.
- *gztar*: gzip'ed tar-file (if the `zlib` module is available).
- *bztar*: bzip2'ed tar-file (if the `bz2` module is available).
- *xztar*: xz'ed tar-file (if the `lzma` module is available).

- *zstdtar*: Zstandard compressed tar-file (if the compression.zstd module is available).

You can register new formats or provide your own archiver for any existing formats, by using register_archive_format().

```
shutil.register_archive_format(name, function[, extra_args[, description]])
```

Register an archiver for the format *name*.

function is the callable that will be used to create archives. The callable will receive the *base_name* of the file to create, followed by the *base_dir* (which defaults to os.curdir) to start archiving from. Further arguments are passed as keyword arguments: *owner*, *group*, *dry_run* and *logger* (as passed in make_archive()).

If *function* has the custom attribute `function.supports_root_dir` set to `True`, the *root_dir* argument is passed as a keyword argument. Otherwise the current working directory of the process is temporarily changed to *root_dir* before calling *function*. In this case make_archive() is not thread-safe.

If given, *extra_args* is a sequence of `(name, value)` pairs that will be used as extra keywords arguments when the archiver callable is used.

description is used by get_archive_formats() which returns the list of archivers. Defaults to an empty string.

Changed in version 3.12: Added support for functions supporting the *root_dir* argument.

shutil.**unregister_archive_format**(*name*)

Remove the archive format *name* from the list of supported formats.

shutil.**unpack_archive**(*filename*[, *extract_dir*[, *format*[, *filter*]]])

Unpack an archive. *filename* is the full path of the archive.

extract_dir is the name of the target directory where the archive is unpacked. If not provided, the current working directory is used.

format is the archive format: one of “zip”, “tar”, “gztar”, “bztar”, “xztar”, or “zstdtar”. Or any other format registered with `register_unpack_format()`. If not provided, `unpack_archive()` will use the archive file name extension and see if an unpacker was registered for that extension. In case none is found, a `ValueError` is raised.

The keyword-only *filter* argument is passed to the underlying unpacking function. For zip files, *filter* is not accepted. For tar files, it is recommended to use `'data'` (default since Python 3.14), unless using features specific to tar and UNIX-like filesystems. (See Extraction filters for details.)

Raises an auditing event `shutil.unpack_archive` with arguments `filename`, `extract_dir`, `format`.

Warning Never extract archives from untrusted sources without prior inspection. It is possible that files are created outside of the path specified in the *extract_dir* argument, for example, members that have absolute filenames or filenames with “..” components.

Since Python 3.14, the defaults for both built-in formats (zip and tar files) will prevent the most dangerous of such security issues, but will not prevent *all* unintended behavior. Read the Hints for further verification section for tar-specific details.

Changed in version 3.7: Accepts a path-like object for *filename* and *extract_dir*.

Changed in version 3.12: Added the *filter* argument.

`shutil.register_unpack_format(name, extensions, function[, extra_args[, description]])`

Registers an unpack format. *name* is the name of the format and *extensions* is a list of extensions corresponding to the format, like `.zip` for Zip files.

function is the callable that will be used to unpack archives. The callable will receive:

- the path of the archive, as a positional argument;
- the directory the archive must be extracted to, as a positional argument;
- possibly a *filter* keyword argument, if it was given to `unpack_archive()`;
- additional keyword arguments, specified by *extra_args* as a sequence of `(name, value)` tuples.

description can be provided to describe the format, and will be returned by the `get_unpack_formats()` function.

`shutil.unregister_unpack_format(name)`

Unregister an unpack format. *name* is the name of the format.

`shutil.get_unpack_formats()`

Return a list of all registered formats for unpacking. Each element of the returned sequence is a tuple `(name, extensions, description)`.

By default `shutil` provides these formats:

- *zip*: ZIP file (unpacking compressed files works only if the corresponding module is available).
- *tar*: uncompressed tar file.
- *gztar*: gzip'ed tar-file (if the `zlib` module is available).
- *bztar*: bzip2'ed tar-file (if the `bz2` module is available).
- *xztar*: xz'ed tar-file (if the `lzma` module is available).
- *zstdtar*: Zstandard compressed tar-file (if the `compression.zstd` module is available).

You can register new formats or provide your own unpacker for any existing formats, by using `register_unpack_format()`.

Archiving example

In this example, we create a gzip'ed tar-file archive containing all files found in the `.ssh` directory of the user:

```
from shutil import make_archive
import os
archive_name = os.path.expanduser(os.path.join('~', 'myarchive'))
root_dir = os.path.expanduser(os.path.join('~', '.ssh'))
make_archive(archive_name, 'gztar', root_dir)
'/Users/tarek/myarchive.tar.gz'
```

The resulting archive contains:

```
tar -tzvf /Users/tarek/myarchive.tar.gz
drwx------ tarek/staff       0 2010-02-01 16:23:40 ./
-rw-r--r-- tarek/staff     609 2008-06-09 13:26:54 ./authorized_keys
-rwxr-xr-x tarek/staff      65 2008-06-09 13:26:54 ./config
-rwx------ tarek/staff     668 2008-06-09 13:26:54 ./id_dsa
-rwxr-xr-x tarek/staff     609 2008-06-09 13:26:54 ./id_dsa.pub
-rw------- tarek/staff    1675 2008-06-09 13:26:54 ./id_rsa
-rw-r--r-- tarek/staff     397 2008-06-09 13:26:54 ./id_rsa.pub
-rw-r--r-- tarek/staff   37192 2010-02-06 18:23:10 ./known_hosts
```

Archiving example with *base_dir*

In this example, similar to the one above, we show how to use `make_archive()`, but this time with the usage of *base_dir*. We now have the following directory structure:

```
tree tmp
tmp
└── root
    └── structure
        ├── content
        │   └── please_add.txt
        └── do_not_add.txt
```

In the final archive, `please_add.txt` should be included, but `do_not_add.txt` should not. Therefore we use the following:

```
from shutil import make_archive
import os
archive_name = os.path.expanduser(os.path.join('~', 'myarchive'))
make_archive(
    archive_name,
    'tar',
    root_dir='tmp/root',
    base_dir='structure/content',
)
'/Users/tarek/myarchive.tar'
```

Listing the files in the resulting archive gives us:

```
python -m tarfile -l /Users/tarek/myarchive.tar
```

```
structure/content/
structure/content/please_add.txt
```

Querying the size of the output terminal

`shutil.get_terminal_size(fallback=(columns, lines))`

Get the size of the terminal window.

For each of the two dimensions, the environment variable, `COLUMNS` and `LINES` respectively, is checked. If the variable is defined and the value is a positive integer, it is used.

When `COLUMNS` or `LINES` is not defined, which is the common case, the terminal connected to `sys.__stdout__` is queried by invoking `os.get_terminal_size()`.

If the terminal size cannot be successfully queried, either because the system doesn't support querying, or because we are not connected to a terminal, the value given in `fallback` parameter is used. `fallback` defaults to `(80, 24)` which is the default size used by many terminal emulators.

The value returned is a named tuple of type `os.terminal_size`.

See also: The Single UNIX Specification, Version 2, Other Environment Variables

Added in version 3.3.

Changed in version 3.11: The `fallback` values are also used if `os.get_terminal_size()` returns zeroes.

© Copyright 2001 Python Software Foundation.
This page is licensed under the Python Software Foundation License Version 2.
Examples, recipes, and other code in the documentation are additionally licensed under the Zero Clause BSD License.
See History and License for more information.

The Python Software Foundation is a non-profit corporation. Please donate.

Last updated on Nov 09, 2026 (17:42 UTC). Found a bug?
Created using Sphinx 9.1.1.